

RoboPong: A UR5e-Based Cup-Pong Throwing Robot

Ali Ibrahim, Yassin Youssef, Boden Duman, Mikhail Chertilin

RIS Lab, Spring 2026

Group 5

<https://github.com/aliahmedd24/RoboPong>

Abstract—RoboPong is a ROS system built around a Universal Robots UR5e arm, a 3D-printed scoop end-effector, and an overhead RGB camera. It plays a one-sided cup-pong: a person puts a red cup on the table opposite the robot, drops a ball in the scoop, and the robot finds the cup, rotates its base to aim, and swings hard enough to fling the ball in a ballistic arc toward the target. What follows covers the game we set out to build versus the one we shipped, the four-node ROS layout, the perception and motion stack, three of the bigger design calls, and what is left to do.

I. INTRODUCTION

Throwing problems are a good integration test for a manipulator. Perception, planning, and dynamic execution all have to work at once, and a botched throw is obvious to anyone watching. RoboPong strips beer pong down to the part that is actually hard for a robot (find the cup, aim, release) and pins the target to a known plane so the whole thing fits in a semester.

II. GAME DESCRIPTION

A. As Originally Envisioned

The original idea for RoboPong was a full back-and-forth game. Multiple cups arranged on the table in a triangular formation, exactly like real beer pong. The robot scans the table, picks a target cup, aims, throws. If it lands the ball, that cup is removed. Then it is the human's turn to throw at the robot's cups. A scoreboard tracks both sides, and the game keeps running until one side has no cups left. Between turns the robot gives feedback, celebrating a good throw or reacting to a miss, so the interaction feels more like playing against an actual opponent than against a machine. This concept got cut. It was too complex and too ambiguous given the limits of the arm and the safety concerns around throwing a ball at a person across the table.

B. As Finally Implemented

What we actually demoed. The human still stands across from the robot, the cup is still red, but the cup has to sit on a specific table at a specific distance. Lateral position is fine: slide it left, slide it right, the robot aims. Distance is locked. Every throw uses the same pre-tuned motion.

A round starts with a service call. The arm goes to its pickup pose. The robot speaks a line through the commentator ("Load me up."), waits a fixed dwell while a human drops a ball in,

then announces it is targeting the cup and swings. A continuous mode plays rounds back-to-back with a configurable inter-round delay, so an audience does not have to keep re-triggering.

Audio commentary covers every phase (pickup, loading, aiming, throwing, idle, error). A latched status topic mirrors the same state for any operator who would rather watch a terminal. The shipped build does not keep score, does not learn from misses, does not adjust throw distance. Those capabilities are stubbed out in the code base; none of them ran on demo day.

III. SOFTWARE ARCHITECTURE

robopong is one catkin package on ROS 1 Noetic. It depends on the lab's existing `cu_ur5e_bringup` and `cu_ur5e_moveit_config` for the UR5e driver, MoveIt planning context, camera, and RViz. The launch file pulls that hardware stack up and then starts four nodes of our own (Fig. 1).

A. Vision Node

`vision_node` reads the colour stream from the overhead camera and publishes the cup position in the base frame on `/robopong/cup_position` as a `geometry_msgs/PointStamped`. It also puts out a debug image (chosen contour, workspace boundary, cup centroid drawn on a copy of the raw frame) plus a one-line status string (Fig. 2). An RViz mesh marker mirrors the published position so the planner's view of the world matches what we see on screen. The marker subscribes to `/robopong/cup_position` rather than running off the camera callback directly. That way it tracks whoever publishes the topic: the real vision pipeline, a manual `rostopic pub`, sim helpers, whatever.

B. Motion Node

`motion_node` owns the arm. Four `std_srvs/Trigger` services: `/robopong/pick_up`, `/robopong/go_ready`, `/robopong/throw`, `/robopong/plan_only`. The first two are slow, collision-aware MoveIt moves to fixed joint targets. The third is the actual throw: read the latest `cup_position`, work out a shoulder-pan aim angle, generate a 1 ms-resolution time-optimal trajectory, ship it through `move_group.execute`

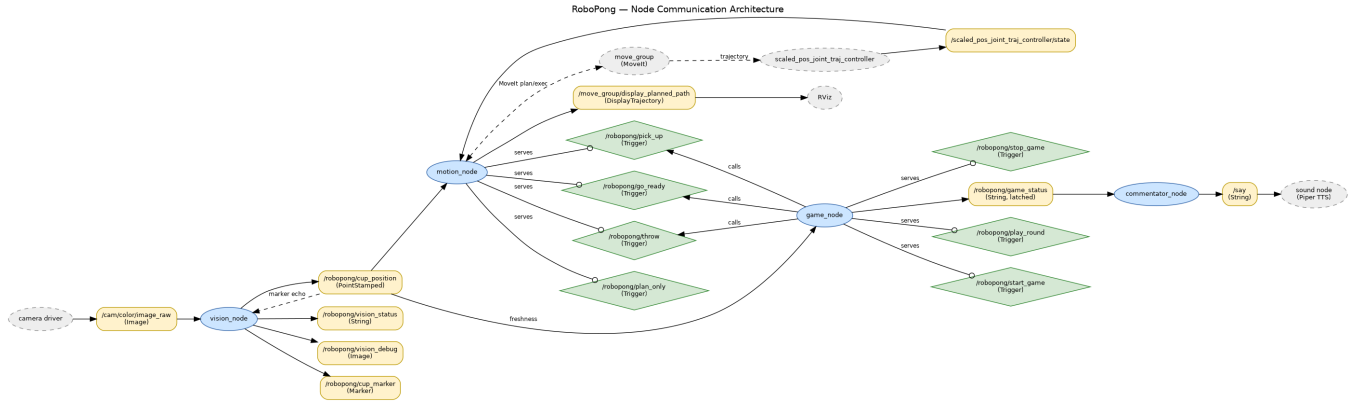


Fig. 1: Full ROS node, topic, and service graph. Blue ovals are nodes implemented in this package (`vision_node`, `motion_node`, `game_node`, `commentator_node`); grey ovals are external nodes brought in by the upstream `cu_ur5e_bringup` stack (`camera_driver`, `move_group`, the scaled-position trajectory controller, `RViz`) and by the operator’s text-to-speech process. Yellow rectangles are topics and green diamonds are services. The vision pipeline streams cup positions left-to-right; `game_node` drives motion_node through the three motion services (`pick_up`, `go_ready`, `throw`) and publishes round-level state on the latched `/robopong/game_status` topic that the commentator consumes.

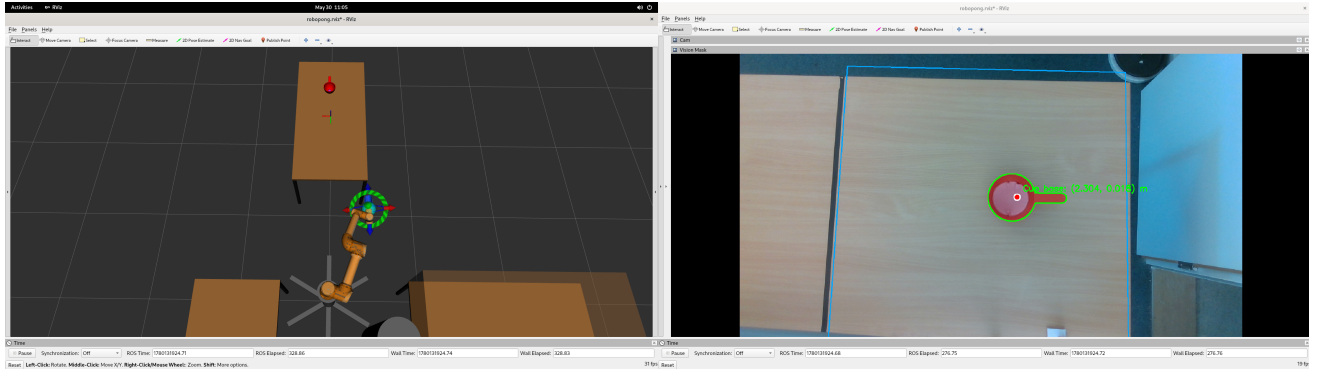


Fig. 2: Operator’s view during a round. Left: the `RViz` scene with the UR5e on its mount, the workspace tables, the green displayed throw trajectory, and the bowl marker that tracks the published cup position. Right: the overhead camera feed with the detected cup contour, centroid, and the workspace boundary overlaid in real time.

so the controller and safety layer see something they recognise. `plan_only` runs the same generation step but only publishes a display trajectory. It is the dry run we used while developing.

Every throw also subscribes to the scaled-position controller’s state topic and dumps the executed trajectory next to the plan: timestamped CSV with positions, velocities, accelerations, plus a 3×3 PNG, both written to `~/robopong_throws/`. This is how we found out the arm was not actually doing what we asked.

C. Game Node

`game_node` runs rounds. `/robopong/play_round` fires a single round. `/robopong/start_game` and `/robopong/stop_game` run them back to back. The body of a round is just the motion services in sequence: `pickup` pose, ball-load dwell, wait for a fresh cup, `throw`. A lock keeps two rounds from racing. Round state is published on the latched `/robopong/game_status` topic, which is what the commentator and any operator display read off. Messages are either bare state names (`IDLE`, `PICKUP`, `AIMING`, `THROWING`) or a state plus a colon-suffixed de-

tail (`LOADING:10.0s`, `ERROR:no fresh cup within 10.0s`). Subscribers that only want the state name split on the first colon, which is what the commentator does.

D. Commentator Node

`commentator_node` maps `game_status` transitions to spoken phrases. Each state has a small pool of lines. On a transition the node picks one at random and pushes it onto a bounded queue. A worker thread drains the queue, publishes on `/say`, and shells out to `espeak-ng`. We deliberately did not go with the more obvious “always speak the latest state.” `AIMING` can resolve in milliseconds if the cup is already in view, and pre-empting whatever phrase was playing turned the audio into stuttering nonsense. Queuing fixes that: every line gets to finish before the next one starts.

E. Communication Summary

Three communication idioms, picked by what the message is for:

- **Topics** for things that stream (`cup_position`, the debug image, the cup marker) or that need to survive late subscribers (`game_status`, latched).

- **Services** when the caller needs an outcome (`pick_up`, `go_ready`, `throw`, `play_round`). The motion services do not return until the motion is done, so `game_node` can call them in sequence without polling.
- **Parameters** for the knobs you set once at launch (camera topic, dwell time, freshness thresholds, table height).

IV. METHODS

A. Cup Detection

Cup detection is classical CV, not learned. Each frame goes from BGR to HSV. Red wraps the hue circle, so we union two ranges (0–10° and 160–180°) into a single binary mask. Open then close with a 5×5 kernel to kill pinholes and bridge small gaps.

We pull contours from the cleaned mask and score them. Largest blob is not enough on its own; we score each contour by $\text{area} \cdot \text{circularity}$, where

$$\text{circularity} = \frac{4\pi \cdot \text{area}}{\text{perimeter}^2}.$$

That penalises anything long and thin (sleeves, a strip of red tape, the operator’s mug) and rewards the cup’s near-circular top-down shape. The chosen contour’s centroid (from image moments) is the cup pixel.

B. Pixel-to-World Mapping

The cup sits on a known horizontal surface, so we do not need 3D ray-casting. A 3×3 homography H maps pixels straight to a workspace-local plane whose origin sits at the centre of the target table. Calibration is a one-time interactive script (`homography_calibration.py`): click the four corners of the rectangular workspace in a fixed order, and `cv2.findHomography` solves for H .

One detail worth recording: we do *not* undistort the input frame. The homography was solved on raw, distorted pixels, so undistorting at runtime would introduce a systematic error against the calibrated H . The price is that H quietly absorbs whatever low-order distortion the lens contributes over the workspace, but that is a constant bias rather than a moving one.

Once the cup pixel is mapped to a workspace-local (x, y) , the node adds the known offset of the workspace centre in the base frame (recorded in `homography.yaml`) and publishes the result. The z comes from a parameter, the table-top height.

C. Smoothing and Outlier Rejection

Detections go through a five-frame rolling average. Anything more than 0.15 m from the buffer mean gets rejected, which kills transient false positives. A hard threshold alone would lock the system out the moment somebody actually moved the cup, so: if no detection has been accepted in over one second, the buffer is cleared and the next detection starts a new average.

D. Wind-Up via MoveIt

MoveIt handles the slow stuff (pickup and aimed-ready) at velocity and acceleration scaling factors of 0.1. These motions cross a cluttered workspace (table, walls, operator clearance volume, tripod) and need collision-aware planning. Their timing is irrelevant to the throw, so MoveIt’s default re-timing is fine.

E. Throw Geometry and Release Speed

We reduced the throw to a planar projectile launch in the sagittal plane through the cup and worked it out on paper before writing any waypoints. The point of doing it this way was to design the three-joint swing against a single number, the scoop-tip speed at release, instead of poking at the executed trajectory and hoping. An earlier round of pure trial and error, fourteen hours of sweeping release poses and angular velocities by hand on the real arm, had not converged on anything that reliably reached the cup. The analytical reduction below is what made the problem solvable.

For a ball launched at height y_0 above the landing point at angle θ above horizontal, range R requires release speed

$$v_0^2 = \frac{g R^2}{2 \cos^2 \theta (y_0 + R \tan \theta)}, \quad (1)$$

where g is gravitational acceleration. Plugging in $R \approx 2$ m (horizontal distance to the cup), y_0 equal to the difference between the scoop-tip height at release and the cup rim, and θ in the energy-optimal band our workspace allowed gives a target tip speed around 3 m/s.

A pure underarm pendulum swing was not an option, because shoulder-lift cannot be driven below horizontal in this mounting. So the swing is a coordinated three-joint whip about shoulder-lift, elbow, and wrist-1 instead. The READY (Fig. 3), RELEASE, and DECEL keyframes introduced in the next subsection are picked so that RELEASE lands at the point of peak tip velocity and zero tip acceleration (the inflection at the top of each driving joint’s bell-shaped velocity profile), and the ball separates from the scoop exactly at peak speed. The numbers: from the cocked start pose (shoulder-lift -3.098 , elbow -2.148 , wrist-1 -0.799 rad), the arm accelerates so that at the release pose (shoulder-lift -2.299 , elbow -1.140 , wrist-1 -2.720 rad) all three driving joints hit their ± 2.5 rad/s peak (positive on shoulder-lift and elbow, negative on wrist-1) with zero acceleration.

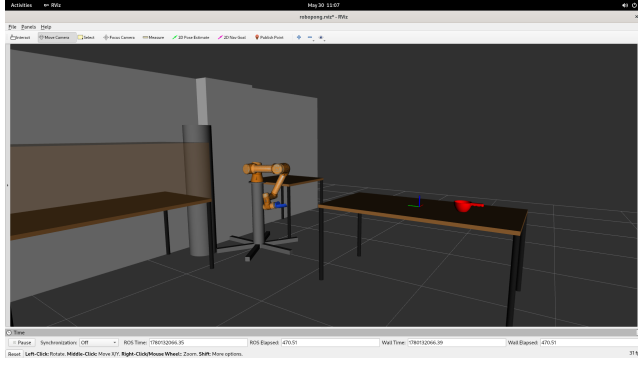


Fig. 3: The READY keyframe: the cocked, aimed pose from which the Ruckig throw segment is launched. The shoulder is rotated back, the elbow folded, and the wrist-1 axis pre-loaded so that the swing toward the RELEASE pose passes through it at peak velocity. The shoulder-pan joint of this pose is shifted by $\text{atan2}(c_y, c_x)$ to aim at the detected cup.

Tip speed at that instant is tied to the joint rates by the planar manipulator Jacobian:

$$v_0 \approx \omega_s(L_1 + L_2) + \omega_e L_2 + \omega_w L_w, \quad (2)$$

where L_1 , L_2 , L_w are the upper-arm, forearm, and wrist-to-tip lengths, and ω_s , ω_e , ω_w are the shoulder-lift, elbow, and wrist-1 angular velocities at release. The release pose is oriented so the tip-velocity vector (tangent to the swept arc) points at the launch elevation θ , with the flat face of the scoop perpendicular to it. Any angular mismatch between the scoop face and the tip-velocity direction either smears the release across several milliseconds or pinches the ball against the scoop edge.

DECEL (shoulder-lift -1.408 , elbow -0.412 , wrist-1 -3.049 rad) is pure follow-through. Ruckig produces a time-optimal, jerk-limited deceleration from the release state back to rest. Without it the controller would have to brake from 2.5 rad/s on three joints to zero in a single time step, which exceeds joint torque limits and trips a protective stop. The post-throw plots (Fig. 5) show planned and executed velocity curves overlapping closely, with the peak landing at the intended release pose and the trajectory tracked cleanly through launch and follow-through.

F. Throw via Time-Optimal Trajectory Generation

With the waypoints from the previous subsection in hand, the trajectory comes out of Ruckig, a real-time time-optimal generator for jerk-limited 6-DoF inputs. Three joint-space keyframes: READY (cocked, aimed), RELEASE (where the ball leaves the scoop), DECEL (follow-through where the arm coasts back to rest).

The throw runs as two Ruckig segments:

- 1) READY $\rightarrow$ RELEASE with a non-zero target velocity. Shoulder-lift, elbow, and wrist-1 each have a target velocity of $\omega = 2.5$ rad/s (with the right sign) and zero target acceleration at the release pose. The arm therefore passes through the release pose mid-ramp, at peak speed.
- 2) RELEASE $\rightarrow$ DECEL, with the *initial* velocity of segment two matching the *final* velocity of segment one.

Target velocity and acceleration are both zero, so the arm coasts back to rest at the follow-through pose.

The two segments are concatenated into a single RobotTrajectory with 1 ms time stamps and handed to `move_group.execute`. The controller brakes at the release pose while the arm still carries substantial velocity; that sharp deceleration is what flings the ball off the scoop into a ballistic arc. Fig. 4 traces the whole computation end to end, from the cup-position read through to the recorded plot.

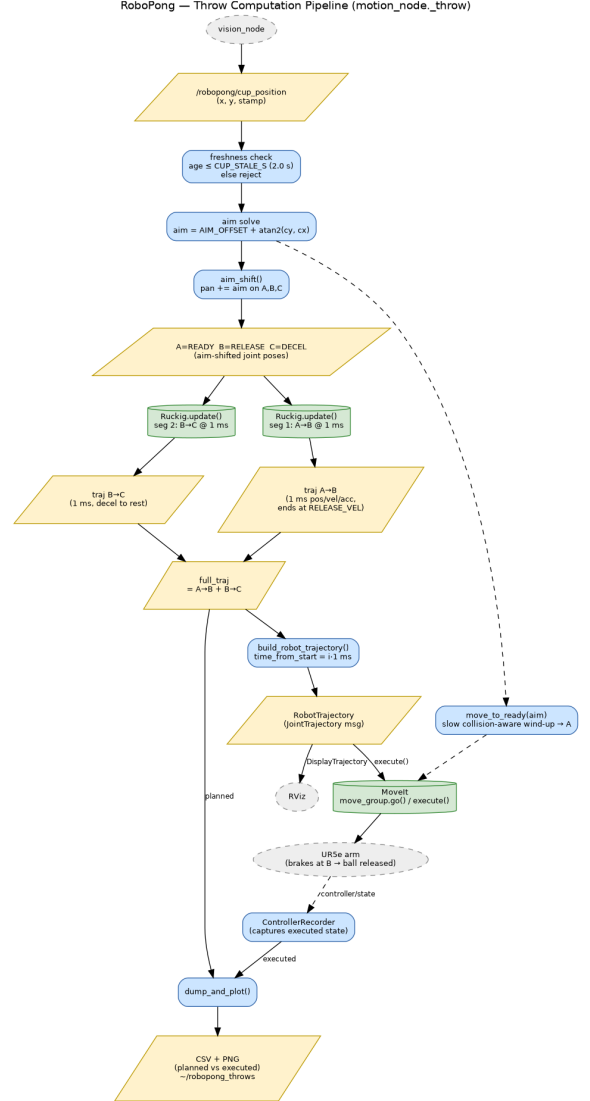


Fig. 4: End-to-end throw computation pipeline inside `motion_node_throw`. A freshness check rejects stale cup positions; the aim solver shifts the shoulder-pan of the three keyframes by $\text{AIM_OFFSET} + \text{atan2}(c_y, c_x)$; Ruckig generates two 1 ms-resolution segments (READY $\rightarrow$ RELEASE ending at the non-zero RELEASE_VEL, and RELEASE $\rightarrow$ DECEL coming back to rest); the concatenated trajectory is wrapped in a RobotTrajectory message and executed through MoveIt's `move_group` (the same channel that drives the slow collision-aware wind-up to READY). A ControllerRecorder subscribes to the controller-state topic during execution and `dump_and_plot` writes a planned-vs-executed CSV and PNG (see Fig. 5) to `~/robopong_throws` on every attempt.

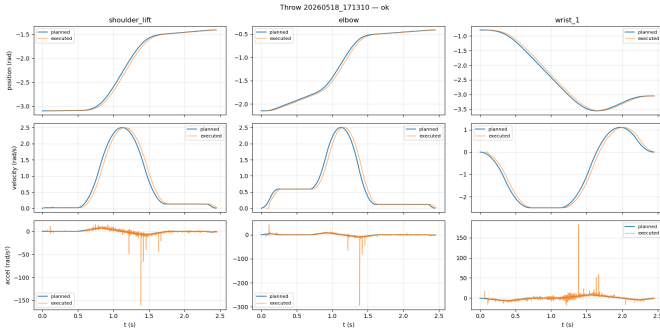


Fig. 5: Planned (blue) versus executed (orange) throw trajectory for the three driving joints (columns: `shoulder_lift`, `elbow`, `wrist_1`; rows: position, velocity, acceleration). The velocity row shows the three bell-shaped profiles all peaking simultaneously near ± 2.5 rad/s at $t \approx 1.2$ s (the instant of ball release predicted by (2)); planned and executed coincide almost exactly through launch and follow-through. The acceleration row’s transients (up to ~ 150 – 300 rad/s²) stay bounded and do not trigger a protective stop, validating the role of the DECEL follow-through waypoint.

G. Aiming

Aim is one degree of freedom. Every waypoint’s shoulder-pan joint is shifted by $\theta = \text{atan2}(c_y, c_x)$ for the latest cup position (c_x, c_y) . The table sits at a fixed distance, so the same throw shape always reaches roughly the same arc length from the base; rotating that arc by θ points it at the cup. This is the biggest simplification we made relative to the original plan, and it is the single largest reason the delivered demo is not the envisioned game.

H. Round Sequencing

A round, as run by `game_node`:

- 1) Call `pick_up`; publish state `PICKUP` (Fig. 6).
- 2) Dwell for `ball_load_delay` seconds; publish `LOADING`.
- 3) Publish `AIMING` and poll `/robopong/cup_position` until a detection arrives whose stamp is within a freshness threshold (default 2 s) of `now`. Abort if no fresh detection arrives within the `cup_wait_timeout`.
- 4) Publish `THROWING`; call `throw`. Motion node re-reads the cup position internally and aims accordingly.
- 5) Publish `IDLE`.

Step 3 is what tells us the cup was not picked up or moved during the load dwell. It is deliberately separate from the cup read inside the throw service: both the orchestrator and the throw bail on a stale cup, so a stale detection has to slip past two checks rather than one.

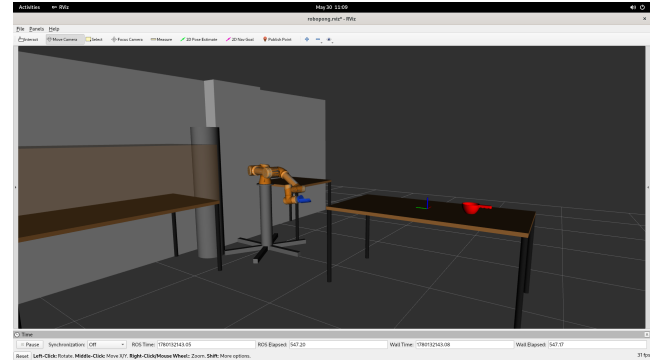


Fig. 6: Robot at the `PICKUP` pose at the start of a round (step 1 above). The scoop is presented in front of the base mount at a height and angle convenient for an operator to drop a ball in; the round dwells here for `ball_load_delay` seconds, then polls cup-position freshness while remaining at pickup before calling the throw service. The aimed wind-up to the `READY` pose happens inside `motion_node._throw` immediately before the Ruckig swing, not as a separate orchestrator step.

V. DESIGN DECISIONS

Three calls made early on that pinned the rest of the project.

A. Static Keyframe Throw vs. Physics-Based Solver

What we did: a fixed pair of keyframes (`RELEASE`, `DECEL`), a hand-tuned release angular velocity $\omega = 2.5$ rad/s, and aim handled by a single rotation of the shoulder-pan joint.

The alternative would have been a physics-based throw solver that takes the cup position and inverts projectile motion to produce release angular velocity, release angle, and shoulder-pan aim. The repo actually contains a scaffold for that approach in `motion_node_v2.py`, including a stub `ThrowSolver` class and a stub `LandingLogger` that would persist landing errors to a calibration file. The v2 node was never wired into the launch file because the solver math was never finished within the project window. It is also worse off than “unfinished”: v2 depends on a `config/throw_profile.yaml` and a `config/solver_data.yaml` that do not exist in the repo, and the v1-fallback path inside v2 raises on the missing profile at `__init__` time. So the scaffold is hidden behind those missing files as well as the missing solver math.

The reason we picked the static path is simple. The MVP requirement was “land the ball in a cup across the table”, and the opponent table is fixed in the world and modelled in the URDF. Throw distance is therefore a constant from the robot’s point of view, so one tuned throw reaches the cup and aiming collapses to a single base rotation. The static path also plotted and debugged easily, with half-hour iteration cycles in the lab. A solver would have piled perceptual, kinematic, and dynamic uncertainty onto every test, all of it useful for the eventual product, terrible on the path to a demo.

B. Classical Colour Segmentation vs. Learned Detection

For perception we went with classical CV: HSV thresholding on two red ranges, morphological cleanup, and a circularity-weighted area score over the contours. A learned detector (YOLO-class) or a Hough-circle + colour post-filter

would have generalised to cups of any colour, but lab lighting was controlled and we got to pick the cup colour anyway. We picked red because it separates cleanly from the table and the robot’s grey mount in HSV. The classical pipeline runs in real time on a laptop without a GPU, and you can debug it by looking at the published `vision_debug` image. A learned detector that mis-fires on demo day is much harder to fix.

C. MoveIt + Ruckig Hybrid vs. Pure MoveIt

For motion we split the tools: MoveIt for the slow, collision-aware moves (pickup, aimed-ready); Ruckig for the throw. The Ruckig output is wrapped in a `RobotTrajectory` and shipped back through `move_group.execute`, so the controller still sees a normally-structured trajectory.

The other option was to use MoveIt for the throw too, with high velocity- and acceleration-scaling factors, and trust its default time-parameterisation to produce a fast enough swing. A throw is not a point-to-point motion, though: it needs a specified *non-zero velocity at an intermediate pose* (the release pose). MoveIt’s time-parameterisation post-processors (IPTP, TOTG) take a geometric path and a velocity profile and *return to rest at every waypoint*, which is exactly the wrong behaviour for a throw. Ruckig accepts a non-zero target velocity directly as input. Splitting the responsibilities gave us each tool in the regime it was actually designed for, while still going through one execution channel (`move_group.execute`) and one set of safety checks.

VI. FUTURE WORK

The distance between the envisioned game and the delivered demo basically reduces to a static throw versus a solved one. Closing it is the main future-work axis, and the code is already structured around it.

A. Variable-Distance Throws

Finish the `motion_node_v2` scaffold by writing the `ThrowSolver`. The cleanest starting point is the lookup mode: a YAML table mapping cup-distance to release angular velocity, built by sweeping ω in the lab and recording where the ball lands. The scaffold already plumbs the solver result out to a debug topic and clamps the resulting speed scale to a safety window. Only the solver itself is missing.

B. Closed-Loop Aim Trim

`LandingLogger` in v2 is the hook for adaptive correction. Every throw would record cup position, commanded speed scale, and observed landing (x, y) . Over many throws the solver could fit a residual model or apply per-cell offsets. The harder piece is observing the landing point automatically. The overhead camera already covers the operator table, so the same vision pipeline could be extended with a second colour band for the ball.

C. Multiple Cups and Target Selection

The vision pipeline already finds and scores multiple candidate contours and currently returns only the best. Returning the ranked list and exposing a target-selection service would enable a classic-pong game mode where the robot is told (or decides) which cup to aim at next. The marker visualiser currently handles only one cup, so this also needs a one-line change to namespace markers by index.

D. Automated Ball Loading

`ball_load_delay` is just a placeholder for a “ball is in the scoop” signal. Sticking a small reflective marker on the ball and reading it with a downward IR sensor on the scoop would let `game_node` drop the fixed dwell and fire the throw as soon as the ball is detected. Continuous play would feel a lot snappier.

E. Score Keeping and Game Mode

The envisioned game has rounds and a score. `game_node` already publishes a structured state machine; a scorekeeping node that subscribes to `game_status` and the landing observations would close the loop into a full match. The commentator can pick that up with score-aware phrases.

VII. LESSONS LEARNED

A. Design

Scope was the largest single risk. Starting from the envisioned game, the obvious decomposition was perception, solver, execution. The solver branch absorbed roughly two weeks of design work that did not contribute to the final demo, and the v2 motion node is what is left of it. In hindsight we should have time-boxed the solver work explicitly: ship the static-throw MVP first, then add the solver as a strict superset, gated by a parameter and with the static path always available as fallback. The v2 scaffold is *shaped* that way (it has a `fallback_to_v1` flag), but we built the scaffold before we built the MVP, which inverted the dependency.

Failure modes deserve as much design as success modes. The single most useful piece of infrastructure was the planned-vs. executed plot dumped on every throw. It was added in week three, and it would have saved a week of guesswork had it been there in week one.

B. The Hidden Velocity Cap

The single largest schedule loss in the project came from a vendor safety limit that we did not know existed. The throw needed peak joint velocities of 2.5 rad/s on three driving joints, but the arm refused to exceed roughly 1.5 rad/s no matter what we commanded. Fig. 7 shows the diagnostic plot from one of these capped attempts: the planned (blue) velocity curves still call for the designed ± 2.5 rad/s peaks on `shoulder_lift`, `elbow`, and `wrist_1`, but the executed (orange) curves clamp cleanly near ± 1.5 rad/s and stretch the position curves out over an extra ≈ 0.4 s. The acceleration row shows no protective-stop event; the controller is simply being told, from somewhere underneath us, that it cannot go faster.

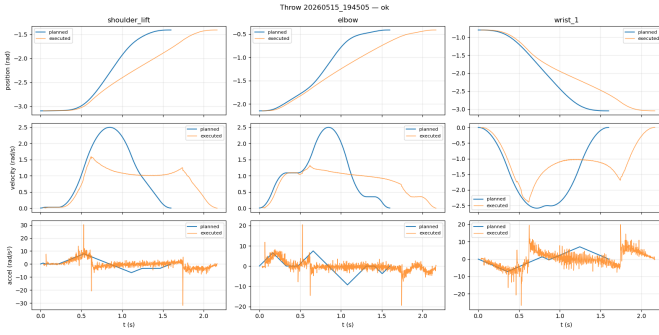


Fig. 7: Diagnostic plot from a throw attempt taken before the teaching-pendant velocity limit was raised. Planned (blue) velocity peaks at the designed ± 2.5 rad/s on all three driving joints, but the executed (orange) curves are clamped cleanly at $\approx \pm 1.5$ rad/s and the position curves lag correspondingly. The clean flat ceiling with no controller error and no protective-stop event is the signature of a vendor safety limit. Contrast with Fig. 5, which is the same plot type taken after the limit was raised and the planned and executed curves overlap.

We chased the cap for several days through every layer of the software stack that we owned: cloning the `joint_limits.yaml` out of the upstream `cu_ur5e` description package into our own configuration and bumping the velocity ceilings, raising the per-joint `max_velocity` limits passed to Ruckig inside `motion_node`, and inspecting the controller-state topic to confirm that the trajectory we sent was indeed time-stamped to require 2.5 rad/s. The controller dutifully clamped the swing every time.

The diagnosis, eventually, was that the cap was not in any of those layers: it had been set on the UR teaching pendant itself, under a safety profile that only an administrator account could edit. Once the professor raised the limit with the appropriate credentials, our existing software immediately produced the intended swing (Fig. 5).

In retrospect the failure mode was identifiable directly from the plot: a clean velocity plateau, position lag without a protective stop, and acceleration transients that stay inside the joint’s normal operating range together rule out torque saturation and planning constraints, and leave a downstream safety clamp as the only consistent explanation. We now treat “the arm refuses to do what the software clearly asks, with no error returned” as a vendor-side hypothesis from the beginning, and would include the teaching pendant’s safety-configuration screen in the hardware checklist for any future UR-based project.

C. Software Implementation

Freshness, not just presence. An early version of `game_node` only checked that *some* cup position had ever been received; this happily threw at a stale detection from minutes earlier when the camera was occluded. Adding a stamp-based freshness check at both the orchestrator and the motion node turned a class of silent failures into loud, observable ones. We now default to freshness checks everywhere a topic crosses a service boundary.

Outlier rejection needs a recovery path. The first jump-rejection filter in `vision_node` was a pure threshold: any

detection more than 0.15 m from the recent mean was discarded. This was correct until the cup was *moved*, after which every real detection was rejected by the stale mean and the system silently went dark. The fix (clear the smoothing buffer if no detection has been accepted for one second) is one of those patches that is obvious in retrospect and impossible to guess in advance. We now write down the filter’s failure modes alongside the filter, not as a follow-up TODO.

Threading and ROS callbacks. The commentator’s first version published the latest state immediately and pre-empted any phrase in flight. On fast state transitions (sub-second AIMING) the audio became unintelligible. Replacing the “latest-wins” model with a bounded queue draining in a worker thread was a one-evening fix; the lesson is that real-time-spoken output is really a producer/consumer problem and benefits from being modelled as one from the start.

D. Group Organisation

Hardware time is the bottleneck. A six-axis arm, a real camera, and a real ball share the lab with other groups. Anything that could be tested in simulation was much cheaper than anything that required hardware, and we under-invested in keeping the simulation path working: by mid-semester the sim branch of our launch file had drifted out of sync with the real one. In future projects, we would treat the simulation entry point as part of the CI loop and re-run it on every push.

Calibration drift is a coordination problem. The homography is invalidated whenever the camera tripod is bumped, which happens roughly once per lab session. Recalibration is fast but it has to be done by whoever is in the lab at the moment, and in early weeks we lost test time to throws that were aiming at a cup the robot couldn’t quite see correctly. Writing a thirty-second “before you trust the cup, run `roslaunch robopong homography_calibration.py`” checklist into the team’s session template removed the failure mode.

Documentation as a coordination tool. The single most useful artifact for working together was the launch file itself. Treating it as the canonical description of how the system is run (including comments on every node about why each parameter has the value it has) meant that a teammate joining a session in progress could be productive within minutes rather than tens of minutes. Future projects will start with a heavily commented launch file.

VIII. CONCLUSION

RoboPong runs the full throwing-task pipeline on real hardware, against a constrained version of the game we set out to build. The constraints we accepted (fixed table distance, fixed throw shape, one cup) bought us a deterministic, debuggable system on a hard schedule. The unfinished v2 scaffold marks where we stopped, not where the design stops; with the solver and landing loop filled in, the same code base would recover most of the original game.